

JavaScript Classes Review

Basics of Working with Classes

- **Definition:** Classes in JavaScript are used to define blueprints for creating objects and encapsulating data. Classes include a constructor which is a special method that gets called automatically when a new object is created from the class. It is used to initialize the properties of the object. The `this` keyword is used here to refer to the current instance of the class. Below the constructor, you can have what are called methods. Methods are functions defined inside a class that perform actions or operations on the class's data or state. They are used to define behaviors that instances of the class can perform.

```
1 class Dog {
2   constructor(name) {
3     this.name = name;
4   }
5
6   bark() {
7     console.log(`${this.name} says woof!`);
8   }
9 }
10
11 const dog = new Dog("Gino");
12 console.log(dog.name); // Gino
```

```
"Gino"
```

To create a new instance of the class, you will use the `new` keyword followed by the class name:

```
const dog = new Dog("Gino");
```

You can also create classes as class expressions. This is where the class is anonymous and assigned to a variable.

```
1 const Dog = class {
2   constructor(name) {
3     this.name = name;
4   }
5
6   bark() {
7     console.log(`${this.name} says woof!`);
8   }
9 };
10
11 const dog = new Dog("Gino");
12 console.log(dog.name); // Gino
13 dog.bark(); // Gino says woof!
```

```
"Gino"
```

```
"Gino says woof!"
```



and methods of another class. Child classes can also extend the functionality of their parent classes by adding new properties and methods. In JavaScript, we use the `extends` keyword to implement inheritance. This keyword indicates that a class is the child class of another class.

<pre>1 class Vehicle { 2 constructor(brand, year) { 3 this.brand = brand; 4 this.year = year; 5 } 6 } 7 8 class Car extends Vehicle { 9 honk() { 10 console.log("Honk! Honk!"); 11 } 12 } 13 14 const myCar = new Car("freeCodeCamp Motors", 2019); 15 console.log(myCar.brand); // freeCodeCamp Motors 16 console.log(myCar.year); // 2019 17 myCar.honk(); // Honk! Honk!</pre>	"freeCodeCamp Motors" 2019 "Honk! Honk!"
---	---

The `super` keyword is used to access the parent class's methods, constructors, and fields.

<pre>1 class Vehicle { 2 constructor(brand, year) { 3 this.brand = brand; 4 this.year = year; 5 } 6 } 7 8 class Car extends Vehicle { 9 constructor(brand, year, numDoors) { 10 super(brand, year); 11 this.numDoors = numDoors; 12 } 13 } 14 15 const myCar = new Car("freeCodeCamp Motors", 2019, 4); 16 console.log(myCar.brand); // freeCodeCamp Motors 17 console.log(myCar.year); // 2019 18 console.log(myCar.numDoors); // 4</pre>	"freeCodeCamp Motors" 2019 4
---	---

Working with Static Methods and Static Properties

- **Static methods:** These methods are often used for utility functions that don't need access to the specific state of an object. They are defined within classes to encapsulate related functionality.

<pre>1 class Movie { 2 constructor(title, rating) { 3 this.title = title; 4 this.rating = rating; 5 } 6 7 static compareMovies(movieA, movieB) { 8 if (movieA.rating > movieB.rating) { 9 console.log(`\${movieA.title} has a higher rating.`); 10 } else if (movieA.rating < movieB.rating) { 11 console.log(`\${movieB.title} has a higher rating.`); 12 } else { 13 console.log("These movies have the same rating."); 14 } 15 } 16 }</pre>	"Movie A has a higher rating."
--	---

Static methods are also helpful for implementing "factory" methods. A factory method is a method that you define in addition to the constructor to create objects based on specific criteria.

```
class Pizza {  
  constructor(type, price) {  
    this.type = type;  
    this.price = price;  
  }  
  
  static createMargherita() {  
    return new this("Margherita", 6.99);  
  }  
}  
  
let myPizza = Pizza.createMargherita();  
console.log(myPizza); // Pizza { type: "Margherita", price: 6.99 }  
console.log(myPizza.type); // Margherita
```

- **Static Properties:** These properties are used to define values or attributes that are associated with a class itself, rather than with instances of the class. Static properties are shared across all instances of the class and can be accessed without creating an instance of the class.

```
1 class Car {  
2   // Static property  
3   static numberOfWheels = 4;  
4  
5   constructor(make, model) {  
6     this.make = make;  
7     this.model = model;  
8   }  
9  
10  // Instance method  
11  getCarInfo() {  
12    return `${this.make} ${this.model}`;  
13  }  
14  
15  // Static method  
16  static getNumberOfWheels() {  
17    return Car.numberOfWheels;  
18  }  
19 }  
20  
21 // Accessing static property directly from the class  
22 console.log(Car.numberOfWheels); // 4
```

4

Assignment

- ☐ Review the JavaScript Classes topics and concepts.

[Submit](#)[Ask for Help](#)